Wi-Fi CSI-Based Gait Recognition Project Guide

Compiled from our guideline chats

March 2026

Table of Contents

1 Purpose

This document compiles the full guidance from our conversation into a downloadable reference for your Intel 5300 Wi-Fi CSI gait-recognition project. It is organized as a practical handbook you can keep beside your setup machine.

2 Executive recommendation

2.1 The short answer

Use the Intel 5300 only as a **dedicated collection platform**, not as your long-term modern research platform.

- **Collector machine OS:** Ubuntu **18.04.0 Desktop** with a **4.15.x** kernel.
- **Do not use the latest Ubuntu** for Intel 5300 CSI collection.
- **Do analysis and modeling on a newer Ubuntu machine**.
- **Use monitor-mode injection** for serious experiments.
- **Preserve raw .dat traces**, then convert them to **HDF5** for long-term research use.

2.2 Why this split is the safest design

The Intel 5300 CSI ecosystem is old but proven. It behaves like a legacy lab instrument: good for measurement, bad for living inside a modern operating system. Keeping collection and analysis separate gives you reproducibility without sacrificing a modern research workflow.

2.3 Main hardware recommendation

Your laptop has a 10th-gen Intel CPU, which is much newer than the kernels typically used by the Intel 5300 CSI toolchain. The cleanest arrangement is:

1. a **collector install** on a separate SSD / partition / spare laptop with Ubuntu 18.04.0 + kernel 4.15,
2. a **modern research machine** for preprocessing, HDF5 conversion, modeling, and paper writing.

3 Operating system decision

3.1 Which Ubuntu version should be used?

Use **Ubuntu 18.04.0 Desktop AMD64 exactly** for the Intel 5300 collection machine.

3.2 Will the setup work on the latest Ubuntu?

No, not reliably.

The Intel 5300 CSI setup depends on an older kernel/driver/firmware stack. A newer Ubuntu release may boot and run, but the CSI-specific wireless driver path is not a dependable target there. For this project, correctness matters more than convenience.

3.3 Practical warning

Do not let the collector machine drift away from the working kernel.

Avoid:

- release upgrades,
- HWE kernel jumps,
- “I updated everything and now CSI disappeared” situations.

Treat the collector machine like a calibrated instrument.

4 Full Intel 5300 collector setup

This section assumes you are building the **collector machine**.

4.1 Step 1 — Verify the machine state

```
uname -r
lspci | grep -i -E 'network|wireless'
sudo lshw -C network
iw dev
mokutil --sb-state
```

4.1.1 Why

- uname -r confirms the active kernel.
- lspci and lshw confirm that the Intel 5300 is detected.
- iw dev shows the interface name.
- mokutil --sb-state tells you whether Secure Boot may interfere with unsigned modules.

4.1.2 Cross-check

You want to see:

- a 4.15.x kernel on the collector install,
- the Intel 5300 listed,
- a wireless interface present,
- Secure Boot disabled if possible.

4.2 Step 2 — Install the required packages

```
sudo apt-get update
sudo apt-get install gcc make linux-headers-$(uname -r) git-core iw
```

4.2.1 Why

You need:

- compiler and build tools,
- exact kernel headers,
- Git for source checkout,
- iw for interface control.

4.2.2 Cross-check

```
gcc --version
make --version
ls -ld /lib/modules/$(uname -r)/build
iw dev
```

4.3 Step 3 — Stop NetworkManager from interfering

First identify the interface, then configure it for manual control.

Example if the interface is wlan0:

```
echo iface wlan0 inet manual | sudo tee -a /etc/network/interfaces
sudo systemctl restart NetworkManager
```

4.3.1 Why

The collector interface should be controlled by the experiment scripts, not constantly reconfigured by the desktop network manager.

4.3.2 Cross-check

```
nmcli dev status
ip link show
```

4.4 Step 4 — Optional module blacklisting

```
echo blacklist iwldvm | sudo tee -a /etc/modprobe.d/csitool.conf
echo blacklist iwlwifi | sudo tee -a /etc/modprobe.d/csitool.conf
```

4.4.1 Why

This is optional, but useful if you want strict control over when the stock wireless stack is loaded.

4.4.2 Cross-check

```
cat /etc/modprobe.d/csitool.conf
```

4.5 Step 5 — Clone the Intel 5300 CSI driver source

Recommended practical path for Ubuntu 18.04 / kernel 4.15:

```
cd ~
git clone https://github.com/spanev/linux-80211n-csitool.git
cd linux-80211n-csitool

export CSITOOL_KERNEL_TAG=csitool-$(uname -r | cut -d . -f 1-2)
echo $CSITOOL_KERNEL_TAG
git checkout ${CSITOOL_KERNEL_TAG}
```

4.5.1 Why

The spanev fork is the practical route for Ubuntu 18.04 with kernel 4.15.

4.5.2 Cross-check

```
git status
git branch --show-current
```

4.6 Step 6 — Build the modified driver

```
make -j"$(nproc)" -C /lib/modules/$(uname -r)/build M=$
(pwd)/drivers/net/wireless/intel/iwlwifi modules
```

4.6.1 Why

This compiles the CSI-enabled iwlwifi module against the exact running kernel.

4.6.2 Cross-check

```
echo $?
find drivers/net/wireless/intel/iwlwifi -name '*.ko' -o -name '*.o' | head
```

4.7 Step 7 — Install the driver

```
sudo make -C /lib/modules/$(uname -r)/build M=$(pwd)/drivers/net/wireless/intel/iwlwifi
INSTALL_MOD_DIR=updates modules_install
sudo depmod
cd ~
```

4.7.1 Why

This installs the updated module into the kernel's updates directory and refreshes module metadata.

4.7.2 Cross-check

```
find /lib/modules/$(uname -r) -path '*updates*iwlwifi*' -type f
modinfo iwlwifi | head
```

4.8 Step 8 — Install the modified firmware and build the logger

```
cd ~
git clone https://github.com/dhalperi/linux-80211n-csitool-supplementary.git

for file in /lib/firmware/iwlwifi-5000-*.ucode; do
  sudo mv "$file" "$file.orig"
done

sudo cp linux-80211n-csitool-supplementary/firmware/iwlwifi-5000-2.ucode.sigcomm2010 /lib/firmware/
sudo ln -sf iwlwifi-5000-2.ucode.sigcomm2010 /lib/firmware/iwlwifi-5000-2.ucode

make -C linux-80211n-csitool-supplementary/netlink
```

4.8.1 Why

The Intel 5300 CSI tool needs the modified firmware and the log_to_file userspace logger.

4.8.2 Cross-check

```
ls -l /lib/firmware/iwlwifi-5000-2.ucode*
ls -l linux-80211n-csitool-supplementary/netlink/log_to_file
```

4.9 Step 9 — Reload the driver with CSI logging enabled

```
export IFACE=$(iw dev | awk '$1=="Interface"{print $2; exit}')

sudo ip link set "$IFACE" down
sudo modprobe -r iwldvm iwlwifi mac80211 || sudo modprobe -r iwlwifi mac80211
sudo modprobe iwlwifi connector_log=0x1
```

4.9.1 Why

This unloads the active wireless stack and reloads the CSI-aware module with connector logging enabled.

4.9.2 Cross-check

```
lsmod | grep -E 'iwlwifi|iwldvm|mac80211'
dmesg | tail -n 50
```

4.10 Step 10 — First smoke test

Start logging:

```
cd ~/linux-80211n-csitool-supplementary/netlink
sudo ./log_to_file ~/csi.dat
```

In another terminal, generate traffic on a compatible open 802.11n HT network:

```
ping -I "$IFACE" <AP_IP_ADDRESS>
```

4.10.1 Why

This is the first proof that CSI is actually being captured.

4.10.2 Cross-check

```
watch -n 1 ls -lh ~/csi.dat
```

If the file grows, the collector path is alive.

5 Controlled monitor-mode collection

For real gait experiments, do **not** rely on random everyday Wi-Fi traffic. Use **controlled monitor-mode injection**.

5.1 Recommended architecture

- **Receiver:** Intel 5300 collector machine
- **Transmitter:** a second Linux machine for controlled packet injection

This makes your dataset reproducible.

5.2 Step 11 — Receiver preflight checks

```
export IFACE=$(iw dev | awk '$1=="Interface"{print $2; exit}')
echo "$IFACE"

mount | grep debugfs || sudo mount -t debugfs none /sys/kernel/debug

uname -r
iw dev
ip link show "$IFACE"
```

5.2.1 Why

You need the correct interface name and a mounted debugfs path for monitor-mode rate control.

5.3 Step 12 — Build the transmitter-side injection tools

On the transmitter:

```
sudo apt update
sudo apt install build-essential libpcap-dev git

cd ~
git clone https://github.com/dhalperi/lorcon-old.git
cd lorcon-old
./configure
make
sudo make install
sudo ldconfig

cd ~
git clone https://github.com/dhalperi/linux-80211n-csitool-supplementary.git
cd linux-80211n-csitool-supplementary/injection
make
```

5.3.1 Why

The injection utilities rely on LORCON v1 and the supplementary injection source.

5.3.2 Cross-check

```
ls -l ~/linux-80211n-csitool-supplementary/injection/random_packets
```

5.4 Step 13 — Put the receiver into monitor mode

On the receiver:

```
cd ~/linux-80211n-csitool-supplementary/injection

grep -n "wlan0" setup_monitor_csi.sh
sed -i "s/wlan0/$IFACE/g" setup_monitor_csi.sh

sudo ./setup_monitor_csi.sh 6 HT20
```

5.4.1 Why

This switches the Intel 5300 receiver into the monitor-mode CSI collection path.

5.4.2 Cross-check

```
iw dev "$IFACE" info
iwconfig "$IFACE"
```

You want monitor mode and the chosen channel.

5.5 Step 14 — Lock the receiver monitor rate

```
echo 0x1c113 | sudo tee
/sys/kernel/debug/ieee80211/phy0/iwlwifi/iwldvm/debug/monitor_tx_rate
```

5.5.1 Why

Fixed-rate experiments are easier to reproduce than adaptive-rate experiments.

5.5.2 Cross-check

```
cat /sys/kernel/debug/ieee80211/phy0/iwlwifi/iwldvm/debug/monitor_tx_rate
```

5.6 Step 15 — Start receiver logging

```
mkdir -p ~/csi_sessions/test01
cd ~/linux-80211n-csitool-supplementary/netlink
sudo ./log_to_file ~/csi_sessions/test01/session_raw.csi.dat
```

5.6.1 Cross-check

```
watch -n 1 ls -lh ~/csi_sessions/test01/session_raw.csi.dat
```

5.7 Step 16 — Put the transmitter into inject mode

On the transmitter:

```
export TX_IFACE=$(iw dev | awk '$1=="Interface"{print $2; exit}')
echo "$TX_IFACE"

cd ~/linux-80211n-csitool-supplementary/injection
grep -n "wlan0" setup_inject.sh
sed -i "s/wlan0/$TX_IFACE/g" setup_inject.sh

sudo ./setup_inject.sh 6 HT20
```

5.7.1 Why

This prepares the second machine for controlled packet injection.

5.7.2 Cross-check

```
iw dev "$TX_IFACE" info
iwconfig "$TX_IFACE"
```

5.8 Step 17 — Fix the transmitter rate and send packets

```
echo 0x4101 | sudo tee "$(find /sys -name monitor_tx_rate | head -n 1)"
sudo ./random_packets 1 100 1
```

5.8.1 Why

This gives you controlled traffic instead of random ambient traffic.

5.8.2 Cross-check

The receiver-side .dat file should grow steadily.

5.9 Step 18 — Stop each trial cleanly

Stop log_to_file with Ctrl+C, then verify:

```
ls -lh ~/csi_sessions/test01/session_raw.csi.dat
```

5.9.1 Why

One session should produce one raw file. Do not merge unrelated trials into the same raw trace.

6 GUI for collection

You asked for a GUI. For the Intel 5300 collector, the most practical design is a **thin GUI wrapper over the working CLI scripts**.

6.1 Why a thin wrapper is the right design

The driver and collection stack are fragile and low-level. Your GUI should not replace them. It should only orchestrate them safely.

6.2 What the GUI should control

At minimum:

- subject ID,
- session ID,
- trial ID,
- room ID,
- path ID,
- walking direction,
- channel,
- bandwidth,
- duration,
- notes.

6.3 Recommended on-disk folder structure

```
csi_sessions/
  subj_001/
    sess_001/
      trial_001_20260319_120000/
        session_raw.csi.dat
```

session.json
end.json

6.4 Lightweight GUI launcher script

A simple Zenity-based launcher is robust on Ubuntu 18.04.

```
#!/usr/bin/env bash
set -euo pipefail

BASE_DIR="$HOME/csi_sessions"
INJ_DIR="$HOME/linux-80211n-csitool-supplementary/injection"
LOG_BIN="$HOME/linux-80211n-csitool-supplementary/netlink/log_to_file"

IFACE=$(iw dev | awk '$1=="Interface"{print $2; exit}')
[ -n "${IFACE:-}" ] || { zenity --error --text="No wireless interface found."; exit 1; }

FORM=$(zenity --forms \
  --title="Intel 5300 CSI Collector" \
  --text="Enter session parameters" \
  --add-entry="Subject ID" \
  --add-entry="Session ID" \
  --add-entry="Trial ID" \
  --add-entry="Room ID" \
  --add-entry="Path ID" \
  --add-entry="Direction (L2R/R2L/FWD/BWD)" \
  --add-entry="Channel" \
  --add-entry="Bandwidth (HT20/HT40)" \
  --add-entry="Duration seconds" \
  --add-entry="Notes")

[ $? -eq 0 ] || exit 0

SUBJECT=$(echo "$FORM" | cut -d'|' -f1)
SESSION=$(echo "$FORM" | cut -d'|' -f2)
TRIAL=$(echo "$FORM" | cut -d'|' -f3)
ROOM=$(echo "$FORM" | cut -d'|' -f4)
PATH_ID=$(echo "$FORM" | cut -d'|' -f5)
DIRECTION=$(echo "$FORM" | cut -d'|' -f6)
CHANNEL=$(echo "$FORM" | cut -d'|' -f7)
BW=$(echo "$FORM" | cut -d'|' -f8)
DURATION=$(echo "$FORM" | cut -d'|' -f9)
NOTES=$(echo "$FORM" | cut -d'|' -f10)
```

```
STAMP=$(date +"%Y%m%d_%H%M%S")
OUTDIR="$BASE_DIR/subj_${SUBJECT}/sess_${SESSION}/trial_${TRIAL}_${STAMP}"
mkdir -p "$OUTDIR"

cat > "$OUTDIR/session.json" <<EOF
{
  "subject_id": "${SUBJECT}",
  "session_id": "${SESSION}",
  "trial_id": "${TRIAL}",
  "room_id": "${ROOM}",
  "path_id": "${PATH_ID}",
  "direction": "${DIRECTION}",
  "channel": "${CHANNEL}",
  "bandwidth": "${BW}",
  "duration_s": "${DURATION}",
  "interface": "${IFACE}",
  "notes": "${NOTES}",
  "start_time": "$(date --iso-8601=seconds)"
}
EOF

cd "$INJ_DIR"

if grep -q "wlan0" setup_monitor_csi.sh; then
  sed -i "s/wlan0/$IFACE/g" setup_monitor_csi.sh
fi

sudo ./setup_monitor_csi.sh "$CHANNEL" "$BW"

sudo "$LOG_BIN" "$OUTDIR/session_raw.csi.dat" &
LOGGER_PID=$!

sleep 2
zenity --info --text="Logging started.\n\nOutput: $OUTDIR\n\nClick OK to begin the
walking trial."

sleep "$DURATION"

sudo kill "$LOGGER_PID" || true
wait "$LOGGER_PID" 2>/dev/null || true

cat > "$OUTDIR/end.json" <<EOF
{
```

```
  "end_time": "$(date --iso-8601=seconds)"
}
EOF

zenity --info --text="Collection complete.\n\nSaved in:\n$OUTDIR"
```

7 Best data format for future research

7.1 Recommended storage strategy

Use three layers:

1. **Raw archive:** session_raw.csi.dat
2. **Canonical archive:** session.h5
3. **Derived features:** Parquet or CSV tables

7.2 Why HDF5 is the best canonical format

HDF5 is ideal because it can store:

- packet-level CSI arrays,
- metadata,
- labels,
- experiment conditions,
- compressed structured datasets,
- multiple logical groups in one file.

7.3 Why not CSV for raw CSI?

CSV is fine for tiny demos, but not for full CSI archives. Raw CSI is high-dimensional and packet-based. CSV becomes bloated and awkward very quickly.

7.4 Recommended HDF5 schema

```
/session
  /csi
    data
    amplitude
    phase
  /packet
    timestamp_low
    bfee_count
    rssi_a
    rssi_b
    rssi_c
    noise
```

agc
antenna_sel
length
rate
/labels
subject_id
trial_id
path_id
direction
room_id
date_id
los_flag
footwear
clothing_state
/meta
tool_name
tool_version
kernel_version
interface_name
channel
bandwidth
tx_rx_distance_m
notes

7.5 Practical rule

Never throw away the raw .dat file. Convert it, but preserve it.

8 Raw .dat to HDF5 conversion workflow

8.1 Environment setup on a modern analysis machine

```
sudo apt update
sudo apt install python3 python3-venv python3-pip
python3 -m venv ~/venvs/csi-research
source ~/venvs/csi-research/bin/activate
pip install numpy h5py csikit
```

8.2 Minimal converter script

Save as ~/dat_to_h5.py:

```
#!/usr/bin/env python3
import json
import os
```

```python
import sys
import numpy as np
import h5py

from CSIKit.reader import IWLBeamformReader

def safe_attr(frame, name, default=0):
    return getattr(frame, name, default)

def main(dat_path, meta_path, out_path):
    with open(meta_path, "r", encoding="utf-8") as f:
        meta = json.load(f)

    reader = IWLBeamformReader()
    csi_data = reader.read_file(dat_path, scaled=True)

    frames = csi_data.frames
    if not frames:
        raise RuntimeError("No CSI frames found.")

    first = np.asarray(frames[0].csi_matrix)
    if first.ndim != 3:
        raise RuntimeError(f"Unexpected CSI matrix shape: {first.shape}")

    n_frames = len(frames)
    n_sub, n_rx, n_tx = first.shape

    csi = np.zeros((n_frames, n_sub, n_rx, n_tx), dtype=np.complex64)
    timestamp_low = np.zeros(n_frames, dtype=np.uint32)
    bfee_count = np.zeros(n_frames, dtype=np.uint16)
    rssi_a = np.zeros(n_frames, dtype=np.int16)
    rssi_b = np.zeros(n_frames, dtype=np.int16)
    rssi_c = np.zeros(n_frames, dtype=np.int16)
    noise = np.zeros(n_frames, dtype=np.int16)
    agc = np.zeros(n_frames, dtype=np.int16)
    antenna_sel = np.zeros(n_frames, dtype=np.uint8)
    length = np.zeros(n_frames, dtype=np.uint16)
    rate = np.zeros(n_frames, dtype=np.uint16)

    for i, frame in enumerate(frames):
        csi[i] = np.asarray(frame.csi_matrix, dtype=np.complex64)
        timestamp_low[i] = safe_attr(frame, "timestamp_low", i)
        bfee_count[i] = safe_attr(frame, "bfee_count", i)
```

```
    rssi_a[i] = safe_attr(frame, "rssi_a", 0)
    rssi_b[i] = safe_attr(frame, "rssi_b", 0)
    rssi_c[i] = safe_attr(frame, "rssi_c", 0)
    noise[i] = safe_attr(frame, "noise", 0)
    agc[i] = safe_attr(frame, "agc", 0)
    antenna_sel[i] = safe_attr(frame, "antenna_sel", 0)
    length[i] = safe_attr(frame, "length", 0)
    rate[i] = safe_attr(frame, "rate", 0)

amplitude = np.abs(csi).astype(np.float32)
phase = np.angle(csi).astype(np.float32)

os.makedirs(os.path.dirname(out_path), exist_ok=True)

with h5py.File(out_path, "w") as f:
    g_session = f.create_group("session")
    g_csi = g_session.create_group("csi")
    g_packet = g_session.create_group("packet")
    g_labels = g_session.create_group("labels")
    g_meta = g_session.create_group("meta")

    chunk_frames = min(256, n_frames)

    g_csi.create_dataset(
        "data",
        data=csi,
        chunks=(chunk_frames, n_sub, n_rx, n_tx),
        compression="gzip",
        compression_opts=4,
        shuffle=True,
        fletcher32=True
    )

    g_csi.create_dataset(
        "amplitude",
        data=amplitude,
        chunks=(chunk_frames, n_sub, n_rx, n_tx),
        compression="gzip",
        compression_opts=4,
        shuffle=True,
        fletcher32=True
    )
```

```
        g_csi.create_dataset(
            "phase",
            data=phase,
            chunks=(chunk_frames, n_sub, n_rx, n_tx),
            compression="gzip",
            compression_opts=4,
            shuffle=True,
            fletcher32=True
        )

        g_packet.create_dataset("timestamp_low", data=timestamp_low)
        g_packet.create_dataset("bfee_count", data=bfee_count)
        g_packet.create_dataset("rssi_a", data=rssi_a)
        g_packet.create_dataset("rssi_b", data=rssi_b)
        g_packet.create_dataset("rssi_c", data=rssi_c)
        g_packet.create_dataset("noise", data=noise)
        g_packet.create_dataset("agc", data=agc)
        g_packet.create_dataset("antenna_sel", data=antenna_sel)
        g_packet.create_dataset("length", data=length)
        g_packet.create_dataset("rate", data=rate)

        for k in ["subject_id", "session_id", "trial_id", "room_id", "path_id", "direction"]:
            if k in meta:
                g_labels.attrs[k] = str(meta[k])

        for k, v in meta.items():
            g_meta.attrs[k] = str(v)

        g_meta.attrs["source_dat"] = os.path.abspath(dat_path)
        g_meta.attrs["scaled"] = "True"
        g_meta.attrs["n_frames"] = str(n_frames)
        g_meta.attrs["n_subcarriers"] = str(n_sub)
        g_meta.attrs["n_rx"] = str(n_rx)
        g_meta.attrs["n_tx"] = str(n_tx)

    print(f"Wrote {out_path}")
    print(f"Frames={n_frames}, subcarriers={n_sub}, rx={n_rx}, tx={n_tx}")

if __name__ == "__main__":
    if len(sys.argv) != 4:
        print("Usage: dat_to_h5.py <session_raw.csi.dat> <session.json> <output.h5>")
        sys.exit(1)
    main(sys.argv[1], sys.argv[2], sys.argv[3])
```

8.3 Example conversion command

```
source ~/venvs/csi-research/bin/activate

python ~/dat_to_h5.py \
  ~/csi_sessions/subj_001/sess_001/trial_001_20260319_120000/session_raw.csi.dat \
  ~/csi_sessions/subj_001/sess_001/trial_001_20260319_120000/session.json \
  ~/canonical/subj_001_sess_001_trial_001.h5
```

9 Research roadmap

9.1 A strong publishable direction

Do not stop at “closed-set identification in one room.”

A much stronger research direction is:

cross-environment, open-set Wi-Fi CSI gait identification with a reproducible Intel 5300 collection stack and a future-proof HDF5 dataset release

9.2 Why this is stronger

A weak paper says: - “we got high accuracy on known subjects in one lab.”

A stronger paper says: - “our system generalizes across rooms, days, conditions, and unknown users.”

9.3 Recommended data collection plan

Collect from:

- 20–40 participants minimum,
- multiple days,
- at least 2 rooms,
- LOS and NLOS conditions,
- multiple walking directions,
- speed variation,
- footwear/clothing variation.

9.4 Baseline modeling plan

Start with:

1. handcrafted features + SVM / Random Forest,
2. CNN + temporal model,
3. transformer-based temporal model.

Then move toward:

- domain generalization,
- open-set recognition,
- calibration and uncertainty.

9.5 Evaluation protocol

Report more than accuracy.

At minimum:

- accuracy,
- macro-F1,
- confusion matrix,
- cross-room performance,
- cross-day performance,
- open-set detection performance,
- ablation studies.

10 Final action plan

10.1 Phase 1 — Build the collector environment

Objective: reach a working Intel 5300 CSI capture path.

Success criteria: csi.dat grows in the smoke test.

10.2 Phase 2 — Build controlled monitor-mode collection

Objective: switch to receiver/transmitter controlled packet collection.

Success criteria: repeatable session-level .dat logs in monitor mode.

10.3 Phase 3 — Add metadata discipline

Objective: every session gets structured metadata and consistent folder layout.

Success criteria: raw files are never anonymous.

10.4 Phase 4 — Add the GUI wrapper

Objective: remove manual collection friction and operator mistakes.

Success criteria: one button-driven collection flow with structured output.

10.5 Phase 5 — Add HDF5 conversion

Objective: convert raw .dat into a reusable scientific archive.

Success criteria: every collected session has raw + JSON + HDF5.

10.6 Phase 6 — Collect a serious dataset

Objective: collect enough subjects, rooms, days, and conditions for a publishable study.

Success criteria: your dataset supports cross-domain experiments, not just in-room demos.

10.7 Phase 7 — Train baselines and advanced models

Objective: move from sanity-check models to a strong final model.

Success criteria: results are stable, reproducible, and supported by ablation studies.

10.8 Phase 8 — Write the paper

Objective: package the system, dataset, experiments, and results into a Q1-level submission.

Success criteria: the paper tells a strong generalization story, not just a local lab story.

11 Appendix — Deep Research prompt

Use this prompt in GPT's Deep Research mode when you want a literature-backed, web-verified version of this same project plan.

I want you to act as a senior WiFi-CSI research engineer and research mentor. I am planning a Q1-level research project on gait recognition / person identification using WiFi CSI data, and my hardware is:

- Processor: Intel i7 10th gen
- NIC card: Intel 5300
- SSD: 256 GB

Your job is to produce a complete, first-to-last, deeply researched guide for this project.

Important instructions:
1. You must actively browse the web and read relevant official documentation, GitHub repositories, and research papers during your response.
2. If any important claim depends on a paper, open the paper or its official abstract / publisher / arXiv version and use it in the answer.
3. Do not just list paper titles. Read enough of the actual paper content to extract methods, limitations, datasets, evaluation protocols, and research gaps.
4. Use citations throughout the answer for all major factual claims.
5. If something is uncertain, outdated, or version-sensitive, verify it by browsing before answering.
6. Do not give a shallow summary. I want a detailed, practical, technically correct roadmap.
7. Do not ask me follow-up questions unless absolutely necessary. Make grounded

assumptions and continue.

I want the final answer to be organized into the following sections:

SECTION A — Executive recommendation
- Tell me whether Intel 5300 is still a good choice in 2026 for WiFi-CSI gait research.
- Compare Intel 5300 with newer CSI-capable platforms if relevant.
- Tell me whether I should use my main machine or a separate collector machine / SSD / dual-boot setup.
- Clearly state the best Ubuntu version for the Intel 5300 collection machine.
- Clearly state whether the latest Ubuntu will work or not, and why.

SECTION B — Full environment setup for Intel 5300 CSI data collection
I need a step-by-step setup guide from zero to working CSI collection.

For every step:
- Give the exact Ubuntu commands.
- Explain why each command is needed.
- Give cross-check / verification commands after each step so I can confirm the environment is correct before moving on.
- Tell me the expected output or expected state.
- Warn me about common failure points and how to debug them.

This setup section must include:
- OS version choice
- kernel version constraints
- driver setup
- firmware replacement if needed
- module loading
- monitor mode / injection mode if relevant
- logging pipeline
- how to test whether CSI is actually being captured
- how to avoid breaking the working setup later

SECTION C — Practical collection architecture
Design the actual collection system for a publishable gait dataset.

Include:
- whether Intel 5300 should be receiver only
- whether I need a separate transmitter
- recommended network topology
- AP/client mode vs monitor/injection mode
- controlled experiment design for gait collection

- room setup
- LOS/NLOS conditions
- channel and bandwidth choices
- repeatability strategy
- session metadata to record

SECTION D — GUI requirement
I need a GUI so I can control collection parameters during data collection.

Please propose the best practical GUI architecture for this Intel 5300 setup, including:
- whether to build a thin wrapper over the CLI tools
- what parameters the GUI should expose
- how the GUI should start/stop collection
- how the GUI should write session metadata
- whether PyQt / Zenity / another lightweight option is best
- a recommended folder structure for saved sessions

If useful, include a minimal but practical implementation plan.

SECTION E — Best data format for future research
I want the collected data stored in the best format for future research use.

You must:
- compare raw .dat, CSV, HDF5, Parquet, and any other relevant format
- recommend the best archival format and explain why
- recommend the best format for derived features
- propose a full HDF5 schema for WiFi CSI gait data
- explain how to preserve raw data, metadata, labels, and experiment conditions
- explain how to support future preprocessing and model training

SECTION F — Research-paper-grounded pipeline for a Q1-level study
This is very important.

Read relevant recent papers on:
- WiFi CSI gait recognition
- WiFi sensing for gait / person identification
- cross-domain / cross-environment WiFi sensing
- open-set gait recognition if relevant
- domain generalization / domain adaptation for WiFi CSI
- benchmark datasets such as Widar / SenseFi / other relevant datasets

Then produce a research-grounded pipeline that includes:
- dataset design

- preprocessing
- segmentation
- feature representations
- baseline models
- strong modern models
- evaluation metrics
- train/validation/test protocols
- cross-room / cross-day / cross-subject testing
- open-set evaluation if appropriate
- ablation studies
- reproducibility checklist

For each major recommendation, tie it back to current literature and cite the papers.

SECTION G — Best publishable research direction
After reading the literature, propose 3 strong paper ideas for me.

For each idea include:
- novelty
- difficulty
- data requirements
- expected contribution
- key baselines
- risks
- target Q1 journals
- why it could be publishable

Then choose the single best one for my hardware and situation.

SECTION H — Q1 journal targeting
Recommend the best realistic Q1 journals for this project.
For each journal:
- explain the fit
- explain what kind of contribution it expects
- tell me what would make my work too weak for that journal
- tell me what would strengthen the paper

SECTION I — Final actionable roadmap
End with a practical roadmap in phases:
Phase 1: environment setup
Phase 2: first CSI capture
Phase 3: stable controlled collection
Phase 4: GUI and data management

Phase 5: dataset collection
Phase 6: preprocessing and baselines
Phase 7: advanced model
Phase 8: paper writing and submission

For each phase:
- objectives
- deliverables
- pitfalls
- success criteria

Additional constraints:
- Prefer official documentation and primary sources when possible.
- When using research papers, prefer recent and high-quality papers, but also include foundational Intel 5300 CSI references if necessary.
- If a tool, repo, or setup is outdated, say so clearly.
- If my 10th gen Intel laptop may have compatibility issues with old Ubuntu / old kernels, analyze that explicitly and suggest the safest workaround.
- Be honest when something is fragile or legacy.
- I care more about correctness and reproducibility than convenience.
- Make the answer detailed and technically rigorous.